

# UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital  
Carrera de Ingeniería en Software



## GUÍA Y EVIDENCIA DE EXPOSICIÓN–DEMOSTRACIÓN

### Generación de código fuente a partir de modelos UML mediante StarUML

#### Proyecto Fin de Curso

Sistema de Gestión para un Gimnasio

Subsistema: Gestión de clientes, membresías y pagos

**Asignatura:** Ingeniería de Requisitos (ISR-401)  
**Docente:** Ing. Gleiston Guerrero Ulloa, PhD.  
**Periodo académico:** 2026–2027 PPA  
**Paralelo:** 4.º Software “A”

#### Integrantes y roles

| Integrante                       | Rol                                              |
|----------------------------------|--------------------------------------------------|
| Díaz Pontón Steven Santiago      | Pendiente de confirmación del integrante         |
| Escudero Plaza María del Rosario | Pendiente de confirmación de la integrante       |
| Mera Arias Erick Jhair           | Configuración, generación y verificación técnica |
| Mesías Quijiye Jhon Alexander    | Pendiente de confirmación del integrante         |

#### Repositorio público de GitHub

[https://github.com/Erick-Mera/GA\\_Exposicion\\_Segundo\\_Corte](https://github.com/Erick-Mera/GA_Exposicion_Segundo_Corte)

Quevedo – Ecuador  
2026

# Índice

|                                                                           |          |
|---------------------------------------------------------------------------|----------|
| <b>Resumen</b>                                                            | <b>1</b> |
| <b>1 Marco teórico</b>                                                    | <b>1</b> |
| 1.1 Model-Driven Engineering (MDE)                                        | 1        |
| 1.2 Model-Driven Architecture (MDA)                                       | 1        |
| 1.3 Model-Driven Development (MDD)                                        | 1        |
| 1.4 Niveles de abstracción                                                | 1        |
| 1.4.1 Computation Independent Model (CIM)                                 | 1        |
| 1.4.2 Platform Independent Model (PIM)                                    | 2        |
| 1.4.3 Platform Specific Model (PSM)                                       | 2        |
| 1.5 Transformaciones de modelos                                           | 2        |
| 1.5.1 Transformaciones modelo a modelo (M2M)                              | 2        |
| 1.5.2 Transformaciones modelo a texto (M2T)                               | 2        |
| 1.6 Metamodelos                                                           | 2        |
| 1.7 Perfiles UML, estereotipos y valores etiquetados                      | 2        |
| 1.8 Plantillas de generación                                              | 2        |
| 1.9 Forward, reverse y round-trip engineering                             | 3        |
| 1.10 Diagramas UML como origen de generación                              | 3        |
| 1.11 Estado del arte de las herramientas de transformación                | 3        |
| <b>2 Justificación de la selección de StarUML</b>                         | <b>3</b> |
| 2.1 Requisitos y restricciones tecnológicas del PFC                       | 3        |
| 2.2 Criterios de selección de la herramienta                              | 3        |
| 2.3 Compatibilidad de StarUML con el lenguaje y la plataforma objetivo    | 3        |
| 2.4 Licencia y disponibilidad                                             | 4        |
| 2.5 Curva de aprendizaje                                                  | 4        |
| 2.6 Comparación con una herramienta alternativa                           | 4        |
| 2.7 Decisión de selección                                                 | 4        |
| <b>3 Descripción del subsistema del PFC</b>                               | <b>4</b> |
| 3.1 Contexto del Sistema de Gestión para un Gimnasio                      | 4        |
| 3.2 Descripción del subsistema de gestión de clientes, membresías y pagos | 5        |
| 3.3 Actores relacionados                                                  | 5        |
| 3.4 Requisitos funcionales relevantes                                     | 5        |
| 3.5 Procesos principales                                                  | 5        |

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| 3.6      | Alcance del subsistema . . . . .                         | 5         |
| 3.7      | Exclusiones del subsistema . . . . .                     | 5         |
| <b>4</b> | <b>Modelo UML de origen</b>                              | <b>6</b>  |
| 4.1      | Descripción general del modelo . . . . .                 | 6         |
| 4.2      | Diagrama de clases . . . . .                             | 6         |
| 4.3      | Clases del dominio . . . . .                             | 6         |
| 4.4      | Atributos y operaciones . . . . .                        | 7         |
| 4.5      | Relaciones y cardinalidades . . . . .                    | 7         |
| 4.6      | Esteretipos y perfiles utilizados . . . . .              | 7         |
| 4.7      | Validación sintáctica y semántica del modelo . . . . .   | 7         |
| 4.8      | Archivo nativo del modelo en el repositorio . . . . .    | 7         |
| <b>5</b> | <b>Configuración del generador</b>                       | <b>7</b>  |
| 5.1      | Generador o extensión utilizada . . . . .                | 8         |
| 5.2      | Origen y versión del generador . . . . .                 | 8         |
| 5.3      | Lenguaje y plataforma objetivo . . . . .                 | 8         |
| 5.4      | Parámetros de generación . . . . .                       | 8         |
| 5.5      | Directorio de salida y convenciones de nombres . . . . . | 8         |
| 5.6      | Política ante la regeneración . . . . .                  | 8         |
| 5.7      | Decisiones de mapeo UML a código . . . . .               | 9         |
| 5.8      | Archivos de configuración en el repositorio . . . . .    | 9         |
| <b>6</b> | <b>Proceso de generación</b>                             | <b>9</b>  |
| 6.1      | Preparación del entorno . . . . .                        | 9         |
| 6.2      | Apertura y selección del modelo de origen . . . . .      | 9         |
| 6.3      | Ejecución del generador . . . . .                        | 10        |
| 6.4      | Registro o log de salida . . . . .                       | 10        |
| 6.5      | Árbol del proyecto generado . . . . .                    | 10        |
| 6.6      | Artefactos producidos . . . . .                          | 10        |
| 6.7      | Tiempo de generación . . . . .                           | 10        |
| <b>7</b> | <b>Verificación del código generado</b>                  | <b>10</b> |
| 7.1      | Estructura del proyecto generado . . . . .               | 10        |
| 7.2      | Verificación sintáctica . . . . .                        | 11        |
| 7.3      | Compilación del código . . . . .                         | 11        |
| 7.4      | Ejecución de una unidad mínima demostrable . . . . .     | 11        |

|           |                                                                |           |
|-----------|----------------------------------------------------------------|-----------|
| 7.5       | Correspondencia entre modelo y código . . . . .                | 12        |
| 7.6       | Diferenciación entre código generado y código manual . . . . . | 12        |
| 7.7       | Limitaciones del generador . . . . .                           | 12        |
| <b>8</b>  | <b>Cierre del ciclo y roundtrip controlado</b>                 | <b>13</b> |
| 8.1       | Estado inicial del modelo y del código . . . . .               | 13        |
| 8.2       | Cambio realizado en el modelo UML . . . . .                    | 13        |
| 8.3       | Regeneración del código . . . . .                              | 13        |
| 8.4       | Evidencia antes y después . . . . .                            | 13        |
| 8.5       | Trazabilidad modelo a código . . . . .                         | 13        |
| 8.6       | Manejo del código manual ante la regeneración . . . . .        | 13        |
| <b>9</b>  | <b>Reparto del trabajo</b>                                     | <b>14</b> |
| 9.1       | Roles y responsabilidades . . . . .                            | 14        |
| 9.2       | Tareas realizadas por integrante . . . . .                     | 14        |
| 9.3       | Commits atribuibles a cada integrante . . . . .                | 14        |
| 9.4       | Log de aportaciones de GitHub . . . . .                        | 14        |
| 9.5       | Participación en la exposición y demostración . . . . .        | 15        |
| <b>10</b> | <b>Conclusiones</b>                                            | <b>15</b> |
| 10.1      | Valor de MDD para el Proyecto Fin de Curso . . . . .           | 15        |
| 10.2      | Aprendizajes del equipo . . . . .                              | 15        |
| 10.3      | Limitaciones identificadas . . . . .                           | 15        |
| <b>A</b>  | <b>Tabla ampliada de correspondencia modelo–código</b>         | <b>17</b> |
| <b>B</b>  | <b>Fragmentos representativos del código generado</b>          | <b>17</b> |
| <b>C</b>  | <b>Evidencias complementarias</b>                              | <b>18</b> |
| <b>D</b>  | <b>Instrucciones detalladas de reproducción</b>                | <b>18</b> |
| <b>E</b>  | <b>Evidencia de contribuciones en GitHub</b>                   | <b>18</b> |

## Resumen

Este informe presenta la aplicación del Desarrollo Dirigido por Modelos (MDD) al Sistema de Gestión para un Gimnasio. Se seleccionó el subsistema de clientes, membresías y pagos por integrar el registro de clientes, la gestión de planes, la vigencia de membresías y el control de pagos. En StarUML 7.1.0 se construyó y validó un modelo con siete clases y dos enumeraciones. Mediante la extensión C# 0.9.7 se generaron nueve archivos fuente, que se incorporaron a un proyecto de consola .NET 10.0 y se recompilaron con cero errores. La ejecución mínima finalizó con código de salida 0. Finalmente, se añadió la operación `suspender(motivo:string)` al modelo y se regeneró el código, comprobando su aparición en `Membresia.cs`. El repositorio público conserva el modelo, el código y las evidencias disponibles; las contribuciones de los demás integrantes se incorporarán cuando sean reportadas.

**Palabras clave:** MDD, StarUML, UML, generación de código, gimnasio.

## 1. Marco teórico

### 1.1. Model-Driven Engineering (MDE)

MDE considera los modelos como artefactos principales del desarrollo y los utiliza para producir código, configuraciones o documentación mediante transformaciones automatizadas. Su propósito es representar el dominio con mayor nivel de abstracción y reducir la dependencia de detalles tecnológicos [1, 2].

### 1.2. Model-Driven Architecture (MDA)

MDA es la propuesta del Object Management Group para separar la descripción funcional de un sistema de su plataforma de implementación. Esta separación se organiza mediante los niveles CIM, PIM y PSM y facilita la evolución hacia diferentes tecnologías [3].

### 1.3. Model-Driven Development (MDD)

MDD aplica los principios de MDE al ciclo de desarrollo: el modelo orienta la construcción y permite generar total o parcialmente el código. La calidad del resultado depende de la precisión del modelo, de las reglas de transformación y de la capacidad del generador [2].

### 1.4. Niveles de abstracción

#### 1.4.1. Computation Independent Model (CIM)

El CIM describe objetivos, actores y procesos del negocio sin representar la estructura interna del software. En el PFC comprende el registro de clientes, la contratación de membresías y el control de pagos.

### 1.4.2. Platform Independent Model (PIM)

El PIM representa la estructura y el comportamiento sin asociarlos con un lenguaje o plataforma. Las clases `Cliente`, `Membresia` y `Pago` constituyen ejemplos de este nivel [4].

### 1.4.3. Platform Specific Model (PSM)

El PSM incorpora decisiones de implementación, como tipos de `C#`, espacios de nombres y convenciones de `.NET`. Es el modelo más próximo al código generado.

## 1.5. Transformaciones de modelos

### 1.5.1. Transformaciones modelo a modelo (M2M)

Las transformaciones M2M producen un modelo destino desde otro modelo, por ejemplo, de PIM a PSM. QVT define mecanismos estándar para relacionar y transformar modelos conformes con MOF [5].

### 1.5.2. Transformaciones modelo a texto (M2T)

Las transformaciones M2T generan artefactos textuales a partir de un modelo. La producción de archivos `.cs` desde las clases UML es una aplicación de este tipo de transformación, estandarizada por MOFM2T [6].

## 1.6. Metamodelos

Un metamodelo define los elementos y reglas de un lenguaje de modelado. UML se estructura sobre MOF; por ello, el generador puede interpretar clases, atributos, operaciones, asociaciones y enumeraciones de manera consistente [4].

## 1.7. Perfiles UML, estereotipos y valores etiquetados

Los perfiles amplían UML sin alterar su metamodelo. Los estereotipos expresan funciones específicas y los valores etiquetados almacenan propiedades adicionales que un generador puede utilizar. En este trabajo se priorizarán los elementos UML estándar compatibles con la extensión seleccionada.

## 1.8. Plantillas de generación

Una plantilla establece cómo se transforma cada elemento del modelo en texto. Puede definir, por ejemplo, la declaración de una clase, el tipo de un atributo o la representación de una relación múltiple mediante una colección.

## 1.9. Forward, reverse y round-trip engineering

La ingeniería directa genera código desde el modelo; la inversa reconstruye un modelo desde código existente. El roundtrip busca mantener la correspondencia entre ambos. La demostración aplicará ingeniería directa y una regeneración controlada para evidenciar el cambio modelo-código.

## 1.10. Diagramas UML como origen de generación

El diagrama de clases es el origen obligatorio porque contiene los elementos estructurales transformables. Otros diagramas solo se incorporan cuando aportan contexto o comportamiento relevante.

## 1.11. Estado del arte de las herramientas de transformación

La literatura registra numerosas herramientas MDD y retos relacionados con interoperabilidad, escalabilidad y adopción industrial [7, 8]. Por ello, la herramienta debe evaluarse por la trazabilidad y reproducibilidad que ofrece en el contexto concreto del PFC.

# 2. Justificación de la selección de StarUML

## 2.1. Requisitos y restricciones tecnológicas del PFC

El PFC corresponde a una aplicación de escritorio para gestionar un gimnasio, con C#, .NET, Windows Forms, SQL Server Express y Dapper. La herramienta CASE debe modelar UML, conservar el archivo nativo, generar código C# compilable y permitir repetir la generación en Windows [9].

## 2.2. Criterios de selección de la herramienta

La selección consideró compatibilidad con UML y C#, disponibilidad del generador, trazabilidad modelo-código, facilidad de instalación, licencia, curva de aprendizaje y posibilidad de almacenar todos los artefactos en GitHub.

## 2.3. Compatibilidad de StarUML con el lenguaje y la plataforma objetivo

StarUML admite diagramas de clases y amplía sus funciones mediante extensiones [10]. La extensión de C# transforma paquetes, clases, atributos, operaciones, enumeraciones y relaciones en archivos .cs [11]. El resultado será la capa estructural del dominio; Windows Forms, Dapper y SQL Server se completarán manualmente.

## 2.4. Licencia y disponibilidad

StarUML se distribuye para Windows y dispone de descarga y licencia comercial documentadas en su sitio oficial [12, 13]. La extensión de C# usa licencia MIT. El equipo registrará la versión realmente instalada y las condiciones empleadas durante la práctica.

## 2.5. Curva de aprendizaje

La interfaz gráfica, el modelado mediante elementos UML y el menú de extensión permiten aprender el flujo básico en poco tiempo. La mayor atención se concentrará en definir correctamente tipos, visibilidad, firmas, cardinalidades y directorio de salida.

## 2.6. Comparación con una herramienta alternativa

**Cuadro 1:** Comparación resumida de herramientas

| Criterio                  | StarUML                                     | Visual Paradigm                             |
|---------------------------|---------------------------------------------|---------------------------------------------|
| <b>Generación C#</b>      | Mediante extensión oficial.                 | Mediante funciones de ingeniería de código. |
| <b>Archivo nativo</b>     | Proyecto .mdj.                              | Proyecto .vpp.                              |
| <b>Uso en la práctica</b> | Flujo directo y suficiente para el alcance. | Mayor amplitud funcional y configuración.   |
| <b>Evidencia</b>          | Modelo, extensión y salida en GitHub.       | Modelo y salida generada.                   |

Visual Paradigm también ofrece generación y recuperación de código [14]; sin embargo, StarUML cubre el alcance estructural del subsistema con un flujo más breve para la demostración.

## 2.7. Decisión de selección

Se seleccionó StarUML porque satisface el modelado UML, la generación C# y la trazabilidad requerida, y permite entregar el archivo .mdj, la configuración y el código. La decisión se limita al objetivo académico de demostrar el ciclo modelo-código y no implica que genere la aplicación completa del gimnasio.

# 3. Descripción del subsistema del PFC

## 3.1. Contexto del Sistema de Gestión para un Gimnasio

El PFC busca centralizar la información operativa del gimnasio y sustituir registros dispersos de clientes, planes, membresías y pagos. La solución completa contempla interfaz de escritorio, reglas de negocio y persistencia; la demostración MDD se concentra en su estructura de dominio.



### 3.2. Descripción del subsistema de gestión de clientes, membresías y pagos

El subsistema administra los datos del cliente, los planes disponibles, la contratación o renovación de membresías y los pagos asociados. Fue elegido porque conecta varias entidades y reglas esenciales del PFC, por lo que resulta representativo para comprobar la generación de código.

### 3.3. Actores relacionados

Intervienen tres actores: el **administrador**, que configura planes y supervisa operaciones; el **empleado**, que registra clientes, membresías y pagos; y el **cliente**, cuyos datos y transacciones son gestionados por el sistema.

### 3.4. Requisitos funcionales relevantes

**Cuadro 2:** Requisitos funcionales seleccionados

| Código    | Descripción resumida                                         |
|-----------|--------------------------------------------------------------|
| RF-SUB-01 | Registrar y consultar la información de los clientes.        |
| RF-SUB-03 | Administrar planes con nombre, duración, precio y estado.    |
| RF-SUB-04 | Asignar un plan a un cliente mediante una membresía.         |
| RF-SUB-06 | Renovar la membresía y actualizar su vigencia.               |
| RF-SUB-07 | Registrar pagos vinculados con cliente, membresía y usuario. |
| RF-SUB-08 | Emitir el comprobante de un pago confirmado.                 |

### 3.5. Procesos principales

El flujo inicia con el registro del cliente, continúa con la selección del plan y la creación o renovación de la membresía, y finaliza con el registro del pago y la emisión del comprobante. Estos procesos justifican las relaciones entre las clases del modelo.

### 3.6. Alcance del subsistema

El modelo abarca personas, clientes, empleados, planes, membresías, pagos, comprobantes y dos enumeraciones. Se generaron sus estructuras C# y se verificó su correspondencia con los elementos UML.

### 3.7. Exclusiones del subsistema

Quedan fuera de la generación la interfaz Windows Forms, autenticación, inventario, rutinas, reportes, caja, base de datos, consultas SQL y repositorios Dapper. Estas funciones pertenecen al PFC, pero no al alcance estructural de la demostración.

## 4. Modelo UML de origen

### 4.1. Descripción general del modelo

El modelo representa el subsistema de clientes, membresías y pagos y sirve como entrada para generar código C#. Incluye nombres, tipos, visibilidad, firmas, relaciones y cardinalidades de acuerdo con UML 2.5.1 [4]. Por incorporar tipos de C# y decisiones de .NET, se considera próximo a un PSM.

### 4.2. Diagrama de clases

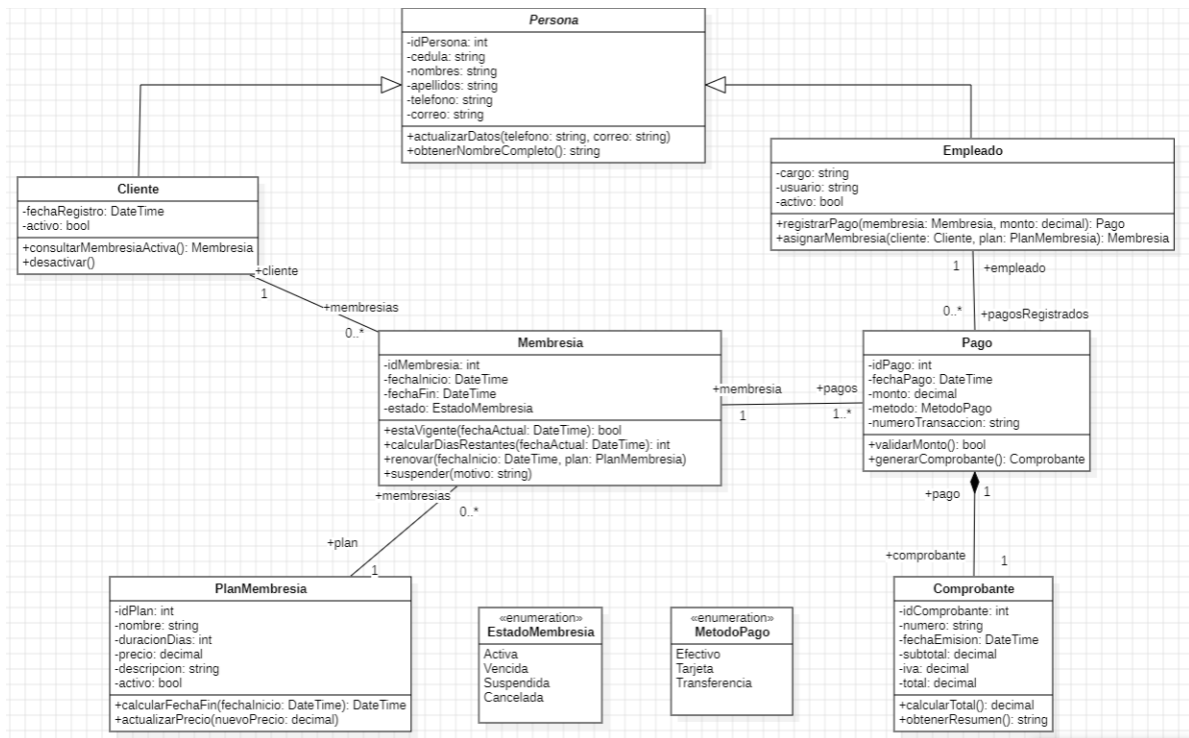


Figura 1: Diagrama de clases del subsistema modelado

### 4.3. Clases del dominio

El modelo contiene siete clases: **Persona**, **Cliente**, **Empleado**, **PlanMembresia**, **Membresia**, **Pago** y **Comprobante**. Además, emplea **EstadoMembresia** y **MetodoPago** para restringir estados y métodos de pago. **Persona** es abstracta y concentra los datos comunes de **Cliente** y **Empleado**. Las responsabilidades detalladas se conservan en el Anexo A.

## 4.4. Atributos y operaciones

Las clases incluyen identificadores y datos propios del dominio, junto con operaciones de actualización, renovación, cálculo, validación y suspensión. Todos los atributos tienen tipo y visibilidad, y las operaciones registran parámetros y retorno. La tabla ampliada se presenta en el Anexo A para evitar duplicar el contenido visible en el diagrama.

## 4.5. Relaciones y cardinalidades

Un cliente puede tener varias membresías; cada membresía corresponde a un plan y contiene uno o varios pagos; un empleado puede registrar múltiples pagos; y cada pago se compone con un comprobante. **Cliente** y **Empleado** heredan de **Persona**. Las multiplicidades visibles en el diagrama se trasladaron a referencias o colecciones tipadas.

## 4.6. Estereotipos y perfiles utilizados

El modelo utiliza elementos estándar de UML compatibles con la extensión de C# [11]. No se añadieron perfiles ni estereotipos específicos porque el generador trabajó directamente con clases, enumeraciones, generalizaciones, asociaciones y composiciones.

## 4.7. Validación sintáctica y semántica del modelo

La revisión comprobó nombres, tipos, visibilidad, firmas, roles y cardinalidades. StarUML 7.1.0 produjo el mensaje *VALIDATION RESULTS — NO ERRORS*, por lo que el modelo quedó validado con cero errores el 18 de julio de 2026.



**Figura 2:** Validación del modelo en StarUML sin errores

## 4.8. Archivo nativo del modelo en el repositorio

El archivo nativo se almacenó como `modelo/modelo_subsistema_gimnasio_corregido.mdj`. Su presencia en la carpeta `modelo/` del repositorio debe verificarse nuevamente antes de la entrega, porque el archivo `.mdj` es la evidencia reproducible del modelo y no puede sustituirse únicamente por la imagen exportada.

# 5. Configuración del generador

### 5.1. Generador o extensión utilizada

Se utilizó *C# Extension for StarUML*, que interpreta el modelo `.mdj` y genera archivos `.cs`. La ejecución se realizó sobre el paquete base del subsistema y no sobre elementos aislados.

### 5.2. Origen y versión del generador

La extensión corresponde al paquete `staruml.csharp`, versión 0.9.7, publicado por Dongjoon Lee con licencia MIT y compatibilidad declarada con StarUML 6 o superior [11]. Se ejecutó en StarUML 7.1.0. Durante la verificación se detectaron tres valores de retorno inválidos para C#: `False`, `return null` en métodos `void` y `return null` para `DateTime`. Se corrigió localmente la plantilla `code-generator.js`; la copia modificada debe conservarse en `plantillas/` junto con una nota que identifique el cambio.

### 5.3. Lenguaje y plataforma objetivo

El lenguaje objetivo es C# y la plataforma de comprobación será .NET sobre Windows. El código generado representará el dominio y posteriormente podrá integrarse con Windows Forms, SQL Server Express y Dapper.

### 5.4. Parámetros de generación

**Cuadro 3:** Configuración resumida del generador

| Parámetro            | Valor                                                    |
|----------------------|----------------------------------------------------------|
| Paquete base         | Gimnasio.Dominio                                         |
| Espacio de nombres   | Gimnasio.Dominio                                         |
| Lenguaje             | C#                                                       |
| Directorio de salida | generado/src/Gimnasio.Dominio/                           |
| Convención           | PascalCase para tipos; camelCase para miembros privados. |
| Sobrescritura        | Permitida solo en el directorio generado.                |

### 5.5. Directorio de salida y convenciones de nombres

Cada clase y enumeración se almacenará en un archivo del mismo nombre. Los archivos manuales de prueba se ubicarán en `generado/verificacion/`, fuera del directorio reemplazable.

### 5.6. Política ante la regeneración

Antes de regenerar se guardó el modelo y se separaron las carpetas `Roundtrip/Antes/` y `Roundtrip/Despues/`. El directorio generado se consideró reemplazable y no recibió lógica manual. Esta política permitió comparar el resultado anterior y posterior sin confundirlo con `Program.cs`.

5.7. Decisiones de mapeo UML a código

Los paquetes se mapearán a espacios de nombres; clases y enumeraciones a tipos C#; atributos a campos; operaciones a métodos; generalizaciones a herencia; y multiplicidades múltiples a colecciones. La tabla ampliada de correspondencia se encuentra en el Anexo A.

5.8. Archivos de configuración en el repositorio

La carpeta plantillas/ conservará configuracion\_generador.md, mapeo\_uml\_cs harp.md y los archivos oficiales de la extensión necesarios para identificar su origen y versión. Esto permitirá repetir el proceso sin depender únicamente de las capturas.

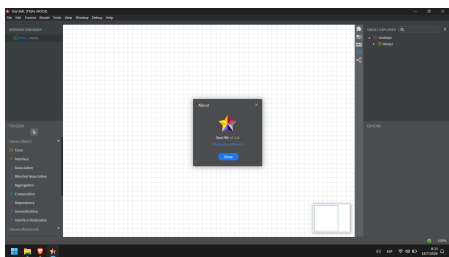
6. Proceso de generación

6.1. Preparación del entorno

La prueba se realizó en Windows 11 de 64 bits con StarUML, la extensión C#, Visual Studio y el SDK de .NET instalados.

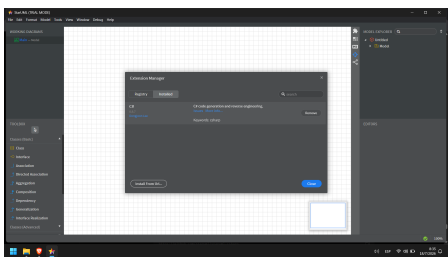
Cuadro 4: Entorno utilizado para la generación

| Componente        | Versión comprobada             |
|-------------------|--------------------------------|
| Sistema operativo | Windows 11, 64 bits            |
| StarUML           | 7.1.0 (modo de prueba)         |
| Extensión C#      | 0.9.7                          |
| Visual Studio     | Community 2026, versión 18.8.0 |
| SDK y plataforma  | .NET SDK 10.0.302; net10.0     |



StarUML 7.1.0

(a)



Extensión C# 0.9.7 instalada

(b)

Figura 3: Versiones de StarUML y de la extensión de generación

6.2. Apertura y selección del modelo de origen

Se abrió el archivo modelo\_subsistema\_gimnasio\_corregido.mdj, se comprobó el diagrama y se seleccionó el paquete Gimnasio.Dominio. El modelo se guardó antes de invocar el generador.

### 6.3. Ejecución del generador

El procedimiento aplicado fue: **1)** abrir **Tools**; **2)** elegir **C# > Generate Code...**; **3)** seleccionar el paquete base; y **4)** indicar **generado/src/Gimnasio.Dominio/** como salida [11].

### 6.4. Registro o log de salida

La extensión no mostró un cronómetro ni produjo un archivo de log exportable. Como registro verificable se conservó la lista de los nueve archivos, sus tamaños y la misma marca temporal de escritura: 18 de julio de 2026 a las 14:57:10. No se atribuyó a StarUML el tiempo de compilación medido por Visual Studio.

### 6.5. Árbol del proyecto generado

El árbol mostró nueve archivos dentro del paquete base: siete clases y dos enumeraciones. **Program.cs** y el archivo de proyecto de la aplicación de consola pertenecen a la verificación manual y no se contabilizaron como generados.

### 6.6. Artefactos producidos

Los archivos producidos fueron los siete tipos del dominio descritos en la Sección 4.3 y las enumeraciones **EstadoMembresia.cs** y **MetodoPago.cs**. No se atribuyeron a StarUML formularios, repositorios, base de datos ni archivos creados manualmente.

### 6.7. Tiempo de generación

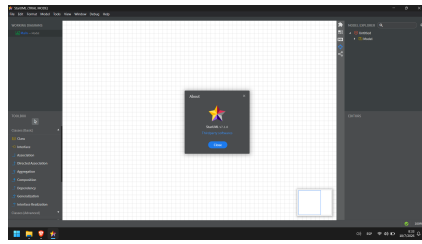
StarUML no expuso el tiempo transcurrido de generación. Los nueve archivos quedaron registrados a las 14:57:10 del 18 de julio de 2026. Visual Studio sí registró una compilación incremental de 0,061 segundos y una recompilación completa de 3,888 segundos; estos valores corresponden a compilación, no a la generación UML-C#.

## 7. Verificación del código generado

La verificación comprobó la estructura, sintaxis, compilación, ejecución y fidelidad del código obtenido después de corregir la plantilla local del generador.

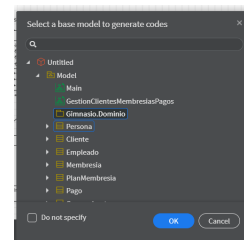
### 7.1. Estructura del proyecto generado

Se verificó la presencia de nueve archivos **.cs**: siete clases y dos enumeraciones, almacenados en **generado/src/Gimnasio.Dominio/**. Los archivos manuales permanecieron en **generado/verificacion/**.

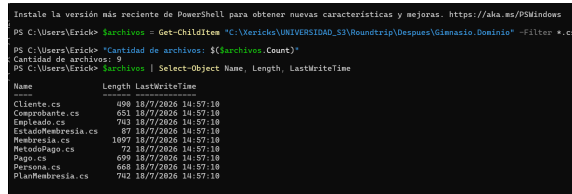


versiones

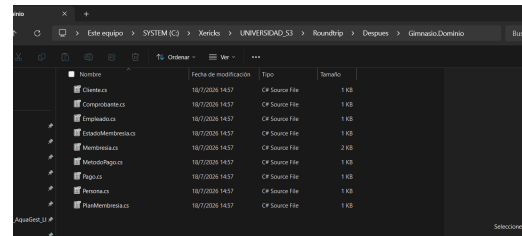
(a) Entorno y



(b) Invocación del generador



(c) Registro de salida



Árbol generado

(d)

**Figura 4:** Evidencias principales del proceso de generación

## 7.2. Verificación sintáctica

La revisión comprobó tipos, firmas, llaves, referencias y colecciones. La primera generación reveló retornos inválidos introducidos por la extensión. Después de corregir la plantilla y regenerar, la recompilación finalizó con cero errores. Visual Studio mostró advertencias CS0169 porque los campos privados del esqueleto todavía no son utilizados; son coherentes con la limitación de un generador estructural y no impidieron la ejecución.

## 7.3. Compilación del código

Como StarUML genera archivos fuente y no una solución ejecutable completa, se usará un proyecto de consola manual que incluya las clases sin modificarlas.

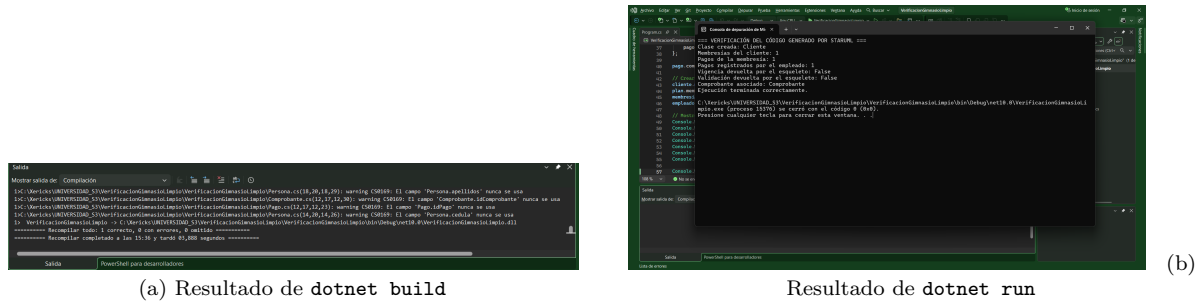
**Listing 1:** Comandos de verificación

```
cd generado/verificacion
dotnet restore
dotnet build
dotnet run
```

**Resultado:** .NET SDK 10.0.302, una recompilación correcta, cero errores y advertencias CS0169 por campos privados aún no utilizados. La recompilación completa registrada duró 3,888 segundos.

## 7.4. Ejecución de una unidad mínima demostrable

El archivo manual `Program.cs` instanció los tipos generados y creó un cliente, una membresía, un pago, un empleado y un comprobante. La consola confirmó las relaciones con valores unitarios, mostró los retornos predeterminados del esqueleto y finalizó con código de salida 0. Esta prueba demuestra integración y ejecución en .NET, pero no constituye la implementación completa de las reglas del gimnasio.



**Figura 5:** Compilación y ejecución del código generado

## 7.5. Correspondencia entre modelo y código

**Cuadro 5:** Correspondencia representativa modelo–código

| Elemento UML              | Código esperado                      | Resultado real                                    |
|---------------------------|--------------------------------------|---------------------------------------------------|
| Persona abstracta         | Clase base abstracta.                | <code>public abstract class Persona</code>        |
| Generalización de Cliente | Herencia de Persona.                 | <code>public class Cliente : Persona</code>       |
| Membresia.estado          | Campo de tipo enumerado.             | <code>private EstadoMembresia estado;</code>      |
| Cliente–Membresía         | Colección tipada.                    | <code>HashSet&lt;Membresia&gt; membresias;</code> |
| 0..*                      |                                      |                                                   |
| Operación de suspensión   | Método público con parámetro string. | <code>public void suspender(string motivo)</code> |
| EstadoMembresia           | Archivo y declaración enum.          | <code>EstadoMembresia.cs</code>                   |

La correspondencia completa de clases, atributos, operaciones y relaciones se traslada al Anexo A.

## 7.6. Diferenciación entre código generado y código manual

Los nueve archivos de `generado/src/` fueron producidos por StarUML. `Program.cs`, el proyecto `.csproj` y los comandos de verificación fueron manuales y permanecieron separados. La plantilla local del generador sí fue corregida antes de regenerar; esto se documenta como una configuración técnica del proceso, no como edición individual de los nueve archivos resultantes.

## 7.7. Limitaciones del generador

La extensión genera principalmente estructura y presenta estas limitaciones:

- no implementa reglas de negocio ni cuerpos funcionales completos;
- no produce Windows Forms, Dapper, SQL Server ni la solución final;
- las asociaciones y colecciones deben revisarse después de generar;
- los cambios manuales dentro de la salida pueden perderse al regenerar.

Estas limitaciones no invalidan la prueba: delimitan qué automatiza StarUML y qué deberá completarse manualmente en el PFC.



## 8. Cierre del ciclo y roundtrip controlado

### 8.1. Estado inicial del modelo y del código

El estado inicial se conservó en Roundtrip/Antes/. En `Membresia.cs` aparecía la operación `renovar(...)` y la clase finalizaba sin una operación de suspensión. El enlace al commit inicial queda pendiente hasta que Erick publique esta evidencia en GitHub.

### 8.2. Cambio realizado en el modelo UML

Se añadió en StarUML la operación `+suspender(motivo:string):void` a la clase `Membresia`. El cambio se realizó en el modelo, sin editar previamente el archivo C# de salida.

### 8.3. Regeneración del código

Después de guardar el `.mdj`, se ejecutó nuevamente el mismo generador sobre el paquete base. La nueva salida se almacenó en Roundtrip/Despues/.

### 8.4. Evidencia antes y después

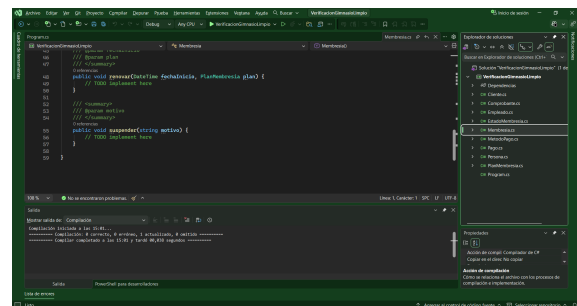
La evidencia compara `Membresia.cs` antes y después de la regeneración. El estado posterior muestra la firma nueva y la recompilación correcta.

```

40      // TODO implement here
41      return 0;
42  }
43
44  /// <summary>
45  /// @param fechaInicio
46  /// @param plan
47  /// </summary>
48  public void renovar(DateTime fechaInicio, PlanMembresia plan) {
49      // TODO implement here
50  }
51
52  }

```

(a) Antes: la clase termina después de `renovar`



(b) Después: aparece `suspender(string motivo)`

**Figura 6:** Evidencia del roundtrip controlado en `Membresia.cs`

### 8.5. Trazabilidad modelo a código

La prueba fue satisfactoria: `Membresia.cs` incorporó `public void suspender(string motivo)` y el proyecto volvió a compilar con cero errores. La correspondencia conserva nombre, visibilidad, parámetro y tipo de retorno definidos en UML.

### 8.6. Manejo del código manual ante la regeneración

El proyecto manual de verificación permanecerá fuera del directorio sobrescrito. Después de regenerar se repetirá `dotnet build` para confirmar que el cambio no afectó su

compilación.

## 9. Reparto del trabajo

### 9.1. Roles y responsabilidades

La distribución prevista contempla cuatro responsabilidades diferenciadas: modelado UML, configuración y generación, verificación técnica, y documentación y presentación. Al cierre de este informe solo se confirmó el aporte técnico de Erick; los roles de los demás integrantes permanecen pendientes de confirmación y deben actualizarse antes de entregar.

### 9.2. Tareas realizadas por integrante

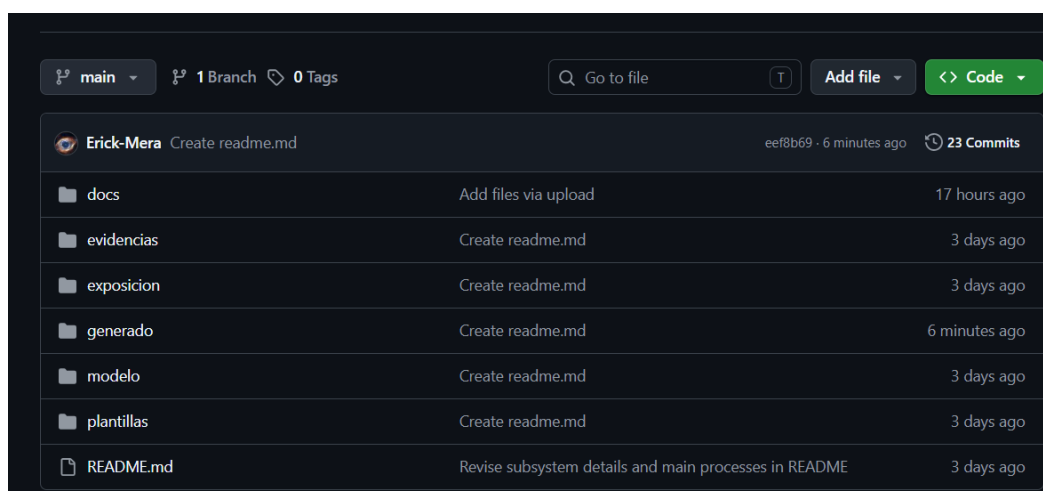
Erick realizó la configuración de StarUML, la corrección de la plantilla del generador, la generación, la verificación en Visual Studio y el roundtrip. Los demás integrantes deberán registrar tareas concretas y verificables, evitando descripciones generales como “ayudó en todo”.

### 9.3. Commits atribuibles a cada integrante

Todavía faltan commits sustantivos identificables de Steven, María y Jhon. También debe publicarse el aporte técnico de Erick mediante su cuenta. Esta sección no se completó con datos inventados porque la rúbrica exige autoría, fecha y enlace verificables.

### 9.4. Log de aportaciones de GitHub

El historial completo se conservará en el Anexo E. Los commits deberán usar la cuenta personal de cada integrante para que la autoría sea identificable.



**Figura 7:** Estructura actual del repositorio público en GitHub

## 9.5. Participación en la exposición y demostración

Los cuatro integrantes explicarán una parte diferenciada y participarán en la demostración: teoría y justificación; modelo; generación y roundtrip; y verificación, resultados y conclusiones.

## 10. Conclusiones

### 10.1. Valor de MDD para el Proyecto Fin de Curso

MDD permitió establecer una relación verificable entre el modelo del subsistema y la estructura inicial del código. Las siete clases y dos enumeraciones produjeron nueve archivos C#, y el roundtrip demostró que una operación añadida al modelo apareció en el archivo esperado. En el PFC este proceso reduce tareas repetitivas si el modelo y la regeneración se controlan.

### 10.2. Aprendizajes del equipo

La actividad permitió diferenciar un diagrama usado solo como documentación de un modelo que actúa como entrada de un generador. También evidenció la necesidad de registrar versiones, parámetros, plantillas modificadas, resultados y commits para que otra persona pueda repetir el proceso.

### 10.3. Limitaciones identificadas

StarUML generó principalmente la estructura del dominio y no reemplazó el desarrollo de interfaces, persistencia ni reglas de negocio. La extensión 0.9.7 requirió una corrección local para emitir valores de retorno válidos en C#. El proyecto recompiló con cero errores, aunque mantuvo advertencias por campos del esqueleto aún no utilizados. Estas limitaciones deben mostrarse en la exposición como parte del resultado real.

## Referencias bibliográficas

- [1] D. C. Schmidt, “Guest editor’s introduction: Model-driven engineering,” *Computer*, vol. 39, pp. 25–31, Feb. 2006.
- [2] B. Selic, “The pragmatics of model-driven development,” *IEEE Software*, vol. 20, no. 5, pp. 19–25, 2003.
- [3] Object Management Group, *Model Driven Architecture Guide, Revision 2.0*. Object Management Group, 2014.
- [4] Object Management Group, *OMG Unified Modeling Language, Version 2.5.1*. Object Management Group, 2017.

- [5] Object Management Group, *Meta Object Facility Query/View/Transformation, Version 1.3*. Object Management Group, 2016.
- [6] Object Management Group, *MOF Model to Text Transformation Language, Version 1.0*. Object Management Group, 2008.
- [7] N. Kahani, M. Bagherzadeh, J. R. Cordy, J. Dingel, and D. Varro, “Survey and classification of model transformation tools,” *Software and Systems Modeling*, vol. 18, no. 4, pp. 2361–2397, 2019.
- [8] A. Bucchiarone, J. Cabot, R. F. Paige, and A. Pierantonio, “Grand challenges in model-driven engineering: An analysis of the state of the research,” *Software and Systems Modeling*, vol. 19, no. 1, pp. 5–13, 2020.
- [9] Microsoft, “C# documentation.” Microsoft Learn, 2026. Consultado el 16 de julio de 2026.
- [10] MKLabs Co., Ltd., “Staruml: A sophisticated software modeler for agile and concise modeling,” 2026. Consultado el 16 de julio de 2026.
- [11] StarUML, “C# extension for staruml,” 2025. Versión 0.9.7; consultado el 16 de julio de 2026.
- [12] MKLabs Co., Ltd., “Download and start now,” 2026. Consultado el 16 de julio de 2026.
- [13] MKLabs Co., Ltd., “Pricing,” 2026. Consultado el 16 de julio de 2026.
- [14] Visual Paradigm International, “Uml/code generation tool,” 2026. Consultado el 16 de julio de 2026.

## A. Tabla ampliada de correspondencia modelo–código

Este anexo conserva el detalle retirado del cuerpo para cumplir el límite de páginas sin perder la definición técnica del modelo.

**Cuadro 6:** Atributos y operaciones principales del dominio

| Clase         | Atributos                                                        | Operaciones                                                    |
|---------------|------------------------------------------------------------------|----------------------------------------------------------------|
| Persona       | idPersona, cedula, nombres, apellidos, telefono, correo          | actualizarDatos(), obtenerNombreCompleto()                     |
| Cliente       | fechaRegistro, activo, membresias                                | consultarMembresiaActiva(), desactivar()                       |
| Empleado      | cargo, usuario, activo, pagosRegistrados                         | registrarPago(), asignarMembresia()                            |
| PlanMembresia | idPlan, nombre, duracionDias, precio, descripcion, activo        | calcularFechaFin(), actualizarPrecio()                         |
| Membresia     | idMembresia, fechaInicio, fechaFin, estado, cliente, plan, pagos | estaVigente(), calcularDiasRestantes(), renovar(), suspender() |
| Pago          | idPago, fechaPago, monto, metodo, numeroTransaccion              | validarMonto(), generarComprobante()                           |
| Comprobante   | idComprobante, numero, fechaEmision, subtotal, iva, total        | calcularTotal(), obtenerResumen()                              |

**Cuadro 7:** Reglas de mapeo UML a C#

| Elemento UML                  | Resultado C#                     | Criterio de comprobación        |
|-------------------------------|----------------------------------|---------------------------------|
| <b>Paquete</b>                | Espacio de nombres y carpeta.    | Coincidencia del nombre base.   |
| <b>Clase</b>                  | Clase y archivo .cs.             | Una declaración por clase.      |
| <b>Atributo</b>               | Campo con tipo y visibilidad.    | Equivalencia de nombre y tipo.  |
| <b>Operación</b>              | Método con parámetros y retorno. | Firma completa.                 |
| <b>Enumeración</b>            | Tipo enum.                       | Valores del modelo conservados. |
| <b>Multiplicidad múltiple</b> | Colección tipada.                | Tipo y cardinalidad revisados.  |
| <b>Generalización</b>         | Herencia mediante dos puntos.    | Clase base correcta.            |

## B. Fragmentos representativos del código generado

Los siguientes fragmentos corresponden a estructuras producidas por StarUML después de corregir la plantilla local. `Program.cs` no forma parte de estos listados.

**Listing 2:** Herencia y colección generadas en `Cliente.cs`

```

1 public class Cliente : Persona {
2     private DateTime fechaRegistro;
3     private bool activo;
4     public HashSet<Membresia> membresias;
5 }

```

**Listing 3:** Operación añadida mediante roundtrip en Membresia.cs

```

1 public void suspender(string motivo) {
2     // TODO implement here
3 }

```

**Listing 4:** Enumeración generada en EstadoMembresia.cs

```

1 public enum EstadoMembresia {
2     Activa, Vencida, Suspendida, Cancelada
3 }

```

## C. Evidencias complementarias

Este anexo reunirá las capturas individuales del entorno, invocación, log, árbol, compilación, ejecución y roundtrip, además de los archivos de registro. En el cuerpo se mantendrán las composiciones necesarias para la evaluación.

## D. Instrucciones detalladas de reproducción

1. clonar el repositorio y abrir el modelo .mdj;
2. comprobar las versiones de StarUML y de la extensión C#;
3. ejecutar el generador con los parámetros documentados;
4. abrir generado/verificacion/;
5. ejecutar `dotnet restore`, `dotnet build` y `dotnet run`;
6. compilar el informe con PdfLaTeX, BibTeX y dos pasadas finales de PdfLaTeX.

## E. Evidencia de contribuciones en GitHub

**Cuadro 8:** Registro ampliado de contribuciones

| Integrante                       | Aporte verificable                                               | Fecha       | Enlace al commit      |
|----------------------------------|------------------------------------------------------------------|-------------|-----------------------|
| Díaz Pontón Steven Santiago      | Aporte no reportado todavía                                      | Pendiente   | Pendiente             |
| Escudero Plaza María del Rosario | Aporte no reportado todavía                                      | Pendiente   | Pendiente             |
| Mera Arias Erick Jhair           | Generación C#, corrección de plantilla, verificación y roundtrip | 18-jul-2026 | Pendiente de publicar |
| Mesías Quijije Jhon Alexander    | Aporte no reportado todavía                                      | Pendiente   | Pendiente             |